



PROGRAMAÇÃO CONCORRENTE

DIEGO KVALEK

CHAPECÓ, SETEMBRO DE 2026

1. Objetivo

O trabalho implementa um sistema de enriquecimento de identificadores com dois processos. O processo pai (P0) inicia e monitora o processo filho (P1). P1 lê uma lista de IDs, distribui os itens entre threads, consulta uma API simulada e grava cada resposta JSON em um arquivo de log. O experimento compara uma thread com quatro threads em três tamanhos de entrada.

2. Implementação

Componente	Responsabilidade
P0 - orquestrador	Cria P1 com <code>subprocess.Popen</code> , aguarda com <code>communicate</code> , mede o tempo, interpreta o código de saída, conta as linhas do log e monta a tabela.
P1 - trabalhador	Cria as threads com <code>threading.Thread</code> , chama a API mockada e registra data, thread, ID e JSON.
Mutexes	Um Lock protege o índice compartilhado e garante distribuição única. Outro Lock protege a escrita concorrente no log.

3. API e formato do log

A API é simulada localmente e inclui uma espera de 5 ms, representando o tempo de uma requisição externa. A resposta segue o formato `{"id":12345,"status":"ok","valor":987.65}`. Cada registro do log fica em uma única linha, no formato definido pelo enunciado.

As listas usadas contêm 50, 200 e 500 IDs. Para cada lista, P0 executa P1 primeiro com uma thread e depois com quatro threads, sempre criando um log separado.

4. Resultados

Lista	IDs	Threads	Tempo (s)	Linhas	Status
Pequena	50	1	1.1158	50	Sucesso
Pequena	50	4	0.1687	50	Sucesso
Media	200	1	1.2119	200	Sucesso
Media	200	4	0.3573	200	Sucesso
Grande	500	1	2.8801	500	Sucesso
Grande	500	4	0.7732	500	Sucesso

Tempos medidos nesta execução. Eles podem variar levemente conforme o computador.

5. Análise de desempenho

O uso de quatro threads reduziu o tempo em todas as escalas. O ganho foi de aproximadamente 6.61 vezes na lista pequena, 3.39 vezes na média e 3.73 vezes na grande. O ganho se aproxima mais de quatro vezes conforme a lista aumenta, porque o custo fixo de criar o processo e as threads passa a representar uma parte menor do tempo total.

Os mutexes não eliminaram o paralelismo porque as regiões críticas são curtas: uma apenas reserva o próximo índice e a outra escreve uma linha no arquivo. A espera da API mockada ocorre fora dos locks, permitindo que outras threads continuem trabalhando. A escrita do log é serializada e pode se tornar um gargalo em volumes muito maiores, mas não limitou de forma relevante estes testes.

6. Auditoria e conclusão

As seis execuções terminaram normalmente, com código de saída zero. Em cada caso, a quantidade de linhas do log foi igual à quantidade de IDs da entrada, portanto nenhum enriquecimento ficou incompleto. O experimento mostra que o paralelismo é vantajoso para uma tarefa dominada pela espera de uma API, desde que o acesso aos dados compartilhados e ao arquivo seja protegido.